Инструкция

Работа рассчитана на 80 минут. Она письменная и индивидуальная. Персональные устройства и интернет не используются, калькулятор, рукописные конспекты разрешены.

Работа состоит из четырёх заданий:

1. Веб-приложение: лабораторная 1, лекции 1-2.
2. Периметр и DDoS: лабораторная 2, лекция 3.
3. Ядро и изоляция: лабораторная 3, лекция 4.
4. SIEM и корреляция: лабораторная 4.

Схемы от руки

• Схемы выполняются от руки, рядом с ответом.
• Обязательно: участники или компоненты, стрелки с подписями (протокол и сообщение), границы доверия пунктиром.
• Для каждого контроля на схеме одна фраза: что он делает и почему стоит именно здесь.
• Схема без подписей или без обоснований получает не больше половины баллов за схему.

В каждом задании

• Схема: элементы, связи, подписи, границы доверия.
• Решение: конкретный механизм, код, значения, команды.
• Обоснование: что решение не закрывает и почему выбран именно этот вариант.

Ответ вида «это защитит от всех атак» баллов не получает. Укажите, от чего защищает элемент и от чего не защищает.

Нулевой вариант: демонстрация

Сценарий

Интернет-магазин «ShopLite». Браузер обращается к API `/api/orders/{id}`. Вход выполняется по паролю, сессия хранится в `cookie auth_token` (JWT, HS256). Перед приложением стоит Nginx с WAF, на хосте работает агент Wazuh.

Задание 1. Веб-приложение (лекции 1-2, лабораторная 1)

Фрагмент сервера:

```
@app.route('/api/orders/<int:order_id>')
@jwt_required
def get_order(order_id):
       user = current_user() # sub из проверенного JWT
       row = db.execute('SELECT * FROM orders WHERE id = ?',(order_id,)).fetchone()
       return jsonify(dict(row))
```

Пользователь с `sub = 42` запрашивает `GET /api/orders/17`. Заказ 17 принадлежит пользователю 7.

a) Нарисуйте схему последовательности атаки. Участники: злоумышленник, WAF, приложение, база данных. Отметьте границу доверия.
b) Назовите уязвимость и объясните, почему WAF её не остановит.
c) Исправьте функцию.

d) Укажите флаги cookie, которых не хватает в ответе `Set-Cookie: auth_token=eyJ...; Path=/`.

Образец ответа

а) Схема

Запрос злоумышленника проходит WAF. Приложение проверяет подпись JWT, но не сравнивает `sub` с владельцем заказа 17, и база отдаёт строку.



Сплошная стрелка: запрос. Пунктирная: ответ. Штрих на стрелке: граница доверия.

б) Уязвимость

BOLA (Broken Object Level Authorization), первая позиция OWASP API Top 10. WAF её не остановит: запрос синтаксически верен, сигнатуры атаки в нём нет, а проверка владельца записи относится к бизнес-логике, которую WAF не знает.

в) Исправление

```
row = db.execute(
       'SELECT * FROM orders WHERE id = ? AND owner_id = ?',
       (order_id, user['sub']),
).fetchone()
if row is None:
       abort(404) # 404, а не 403: не подтверждаем, что чужой заказ существует
```

г) Флаги cookie

Не хватает `HttpOnly`, `Secure` и `SameSite`. Во Flask это выглядит так:

```
resp.set_cookie('auth_token', token, httponly=True, secure=True, samesite='Lax',
path='/')
```

• `HttpOnly`: скрипт не читает cookie через `document.cookie`, поэтому XSS ничего не украдёт. Не защищает от отправки XSS запросов от имени жертвы;

• `Secure`: cookie уходит только по HTTPS, нельзя перехватить в открытой сети. Не защищает от XSS;

• `SameSite=Lax`: браузер не отправит cookie в фоновом кросс-сайтовом POST-запросе, это блокирует классический CSRF. Не защищает от атак с поддомена того же сайта.

Задание 2. Периметр (лекция 3, лабораторная 2)

Перед API стоят CDN и Nginx: `limit_req_zone $binary_remote_addr zone=shop:10m rate=5r/s;` и `limit_req zone=shop burst=10;` для `/api/orders/`. Один адрес присылает 25 запросов одновременно.

• Сколько запросов обработается, сколько получит 503? Объясните алгоритм leaky bucket.

• Какой адрес видит `$binary_remote_addr` за CDN? Что нужно изменить?

• Нарисуйте путь пакета от клиента до приложения и отметьте уровень (L3/L4 или L7) каждого элемента.

Образец ответа

а) Расчёт

Скорость 5 r/s пропускает один запрос каждые 200 мс. Алгоритм leaky bucket работает как ведро: запросы в него поступают, а уходят с постоянной скоростью. Без `nodelay` лишние запросы не отбрасываются сразу, а ждут в очереди длиной `burst = 10`. Из 25 одновременных запросов пройдут 11: первый сразу и ещё 10 из очереди, по одному каждые 200 мс. Последний из них уйдёт через 2 секунды. Остальные 14 получат 503.

С параметром `nodelay` количество тоже будет 11 и 14, но все 11 пройдут сразу, без ожидания.

б) Адрес

За CDN `$binary_remote_addr` содержит IP узла CDN, а не клиента. Все клиенты одного узла делят одно ведро, и один злоумышленник исчерпывает лимит для всех. Нужно взять IP клиента из заголовка, который добавляет CDN, и доверять этому заголовку только с адресов CDN:

```
set_real_ip_from 203.0.113.0/24;  # диапазон IP узлов CDN (пример)
real_ip_header X-Forwarded-For;   # заголовок, который добавляет CDN
real_ip_recursive on;
```

Если доверять заголовку с любого адреса, клиент сам подставит свой X-Forwarded-For и обойдёт лимит. Имя заголовка зависит от CDN, например X-Real-IP или CF-Connecting-IP.

в) Путь пакета

Элементы в порядке прохождения, с уровнем каждого:

1. Клиент: IP-пакет (L3), TCP-сегмент (L4), внутри HTTP-запрос (L7).
2. CDN-узел: принимает TCP-соединение (L3/L4), разбирает HTTP и добавляет X-Forwarded-For (L7).
3. Nginx с limit_req: TCP-соединение от CDN (L3/L4), лимит по ключу `$binary_remote_addr` (L7).
4. Приложение API: обрабатывает только запросы, прошедшие лимит (L7).

Лимит работает на L7, но его ключ берётся из L3-адреса соединения. Поэтому за CDN нужна подмена из пункта б.

Задание 3. Ядро (лекция 4, лабораторная 3)

eBPF-зонд подключён к `sys_enter_openat` и фильтрует запись в каталог `/etc/cron.d/`. Совпавшие события передаются Python-монитору через perf buffer.

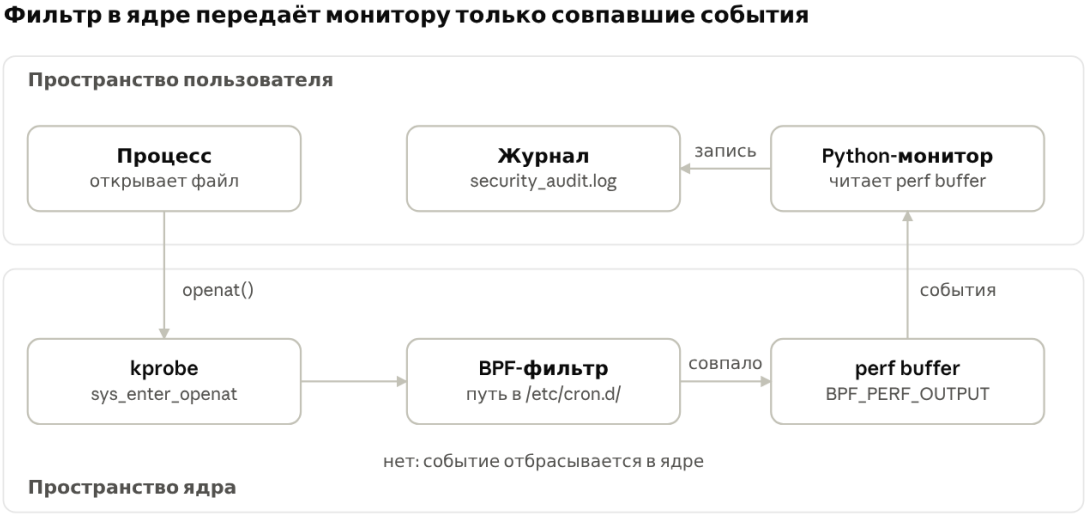
• Нарисуйте схему: пространство пользователя, пространство ядра, kprobe, perf buffer, монитор, журнал.

• Объясните, почему фильтр стоит в ядре.

• Почему имя файла, которое проверил зонд, может отличаться от имени, которое открыло ядро?

Образец ответа

а) Схема.



Событие рождается в процессе, который вызывает `openat()`. kprobe на `sys_enter_openat` срабатывает внутри ядра, BPF-фильтр проверяет путь. Совпавшие события уходят через perf buffer Python-монитору, а монитор пишет журнал. Несовпавшие отбрасываются в ядре.

б) Без фильтра в ядре каждый вызов `openat()` во всей системе копировался бы в пространство пользователя, с переключением контекста на каждое событие. В ядре фильтр отбрасывает лишнее на месте, и наружу уходят только совпадения. Верификатор eBPF гарантирует, что программа безопасна и не зацикливается, но не проверяет, что фильтр правильный.

в) Зонд читает строку пути из пользовательской памяти при входе в системный вызов (`bpf_probe_read_user_str`). Другой поток того же процесса может переписать эту строку по тому же указателю до того, как ядро её использует. Это классический TOCTOU. Кроме того, `openat()` работает с `dirfd`: зонд видит только относительную строку вроде `cron.d/job`, без каталога, а символические ссылки ядро разрешает уже после проверки. Поэтому зонд проверяет строку, а не итоговый файл.

Задание 4. SIEM (лабораторная 4)

В `access.log` трижды за 60 секунд появилась строка с `%27%20OR%201%3D1` от одного адреса.

• Напишите правило уровня 10 с `if_sid 31100` и `match`.

• Опишите правило корреляции: три срабатывания за 120 секунд с одного адреса дают алерт уровня 13.

• Нарисуйте путь события от агента до дашборда.

Образец ответа

а) Правило опирается на `if_sid 31100` (общие события `access.log`) и ищет `match` в строке лога. Wazuh сравнивает текст так, как он записан в `access.log`, поэтому ищется закодированная форма `%27%20OR%201%3D1`, а не `'OR 1=1'`.

```
<group name="web,attack,">
<rule id="100000" level="10">
    <if_sid>31100</if_sid>
    <match>%27%20OR%201%3D1</match>
    <description>SQLi-попытка: ' OR 1=1 в строке запроса</description>
    <mitre>
      <id>T1190</id>
    </mitre>
  </rule>
</group>
```

б) Уровень 13 срабатывает, когда правило 100000 сработало 3 раза за 120 секунд с одного адреса. Строка из условия (3 раза за 60 секунд) укладывается в это окно, поэтому алерт сработает. Правило добавляется в тот же `<group>`:

```
  <rule id="100001" level="13" frequency="3" timeframe="120">
    <if_matched_sid>100000</if_matched_sid>
    <same_source_ip />
    <description>Повторные SQLi-попытки с одного IP: 3 за 120 секунд</description>
    <mitre>
      <id>T1190</id>
    </mitre>
  </rule>
```

в) Порядок элементов:

1. Wazuh-агент читает `access.log` (директива `localfile`) и отправляет новую строку менеджеру.

2. Менеджер декодирует строку декодером веб-логов и применяет правила: 31100, затем 100000 (уровень 10). Правило 100001 смотрит на срабатывания 100000 и даёт уровень 13.

3. Алерты попадают в `alerts.json`, Filebeat передаёт их в индексатор Wazuh.

5. Дашборд Wazuh читает индексатор и показывает алерты.

Вопросы для подготовки

Вопросы охватывают лекции 1-4 и лабораторные 1-4. Ответ должен опираться на механизм, а не на запоминаемое определение.

Лекции 1–2: Zero Trust и веб-приложения

1. Почему периметральная модель перестаёт работать при удалённой работе, SaaS и BYOD?

2. Что делают PEP, PDP, PE и PA в NIST SP 800-207? Кто принимает решение, а кто его исполняет?

3. Чем JIT-доступ отличается от JEA и как каждый из них уменьшает радиус поражения?

4. Назовите три сигнала, по которым PDP пересчитывает Trust Score. Почему поведенческий сигнал (UEBA) может ошибаться?

5. Почему SQL-инъекция возникает, а параметризованный запрос её устраняет? Что делает СУБД с параметрами?

6. Чем in-band, inferential и out-of-band SQL-инъекции отличаются по каналу, по которому извлекаются данные?

7. Почему условие OR 1=1 обходит проверку пароля?

8. Чем reflected XSS отличается от stored и DOM-based XSS? Какой из них не виден на сервере и почему?

9. Почему контекстно-зависимое экранирование надёжнее фильтрации?

10. Почему BOLA нельзя закрыть только случайными UUID?

11. Почему в JWT нельзя кладуть конфиденциальные данные? Чем JWS отличается от JWE?

12. Как работают атаки alg=none и algorithm confusion? Что должен проверять сервер?

13. Что HttpOnly защищает и что не защищает?

14. Что делает SameSite=Lax и почему это не полная защита от CSRF?

15. Почему браузер блокирует CORS, если сервер вернул Access-Control-Allow-Origin: * и Allow-Credentials: true?

16. Как nonce и hash защищают CSP от inline-скриптов? Почему unsafe-inline обнуляет защиту?

17. Чем вложенные запросы GraphQL отличаются от проблемы N+1?

18. Какие ограничения у проверки SQL-запросов через сигнатуры, токенизацию и AST? **(лекция 3)**

Лекция 3 и лабораторная 2: периметр и DDoS

19. Чем stateful-инспекция отличается от NGFW с DPI, и что NGFW не видит в приложении?

20. Сравните негативную и позитивную модели WAF: что блокирует каждая и какой ценой?

21. Как работают SYN cookies: что сервер хранит, а что восстанавливает из ACK?

22. Почему DNS-amplification даёт BAF около 50? Какие протоколы дают больший фактор и почему?

23. Как работает leaky bucket в limit_req? За что отвечают rate и burst?

24. Почему Slowloris исчерпывает потоки, а не канал? Какие директивы Nginx его ограничивают?

25. Как работают BGP diversion и scrubbing-центр? Чем CDN отличается от scrubbing по задаче?

26. Почему SYN cookies не помогают против UDP-рефлексии?

Лекция 4 и лабораторная 3: идентичность, изоляция, DevSecOps

27. Чем namespaces отличаются от cgroups? Что видит процесс в PID namespace?

28. Что меняет user namespace для root в контейнере при побеге?

29. Что проверяет verifier eBPF и почему нельзя просто загрузить свой код в ядро?

30. Чем kprobe отличается от tracepoint по стабильности и назначению?

31. Что такое TOCTOU на примере указателя в системном вызове? Как безопасно прочитать строку по указателю из eBPF-программы?

32. Чем Seccomp-BPF отличается от eBPF kprobe в задаче блокировки системных вызовов?

33. Почему eBPF-зонд обычно используется для аудита, а не для блокировки?

34. Чем OAuth 2.0 отличается от OIDC? Какие поля ID Token проверяет клиент?

35. Какой канал в Authorization Code Flow является back-channel и какая защита на нём обязательна? Когда нужен mTLS?

36. Почему WebAuthn защищает от фишинга за счёт привязки к Origin? Что эта привязка не покрывает?

37. Почему SAST даёт ложные срабатывания, а DAST – меньше? Докажет ли задержка ответа после SLEEP-пейлоада уязвимость без повтора?

38. Что такое SBOM и зачем он нужен при атаке нулевого дня на библиотеку?

Лабораторная 4: SOC и SIEM

39. Из каких этапов состоит конвейер SIEM? Где происходит нормализация, фильтрация и корреляция?

40. Чем корреляция отличается от фильтрации? Приведите пример корреляции по времени и по источнику.

41. Чем IDS отличается от IPS по месту установки? Что видит NIDS при TLS?

42. Чем NTA отличается от NDR?

43. Как Wazuh использует localfile, декодеры, if_sid и match? Почему закодированная строка требует совпадения кодировки?

44. Зачем правилу привязка к MITRE ATT&CK? Какие техники соответствуют SSRF и web-shell?

45. Что такое ложное срабатывание в SOC? Назовите три способа его снизить.